Per-Face Clearance Filter for Robots in Configuration Space

Adapting idea 1 (the per-piece step filter) from rigid bodies to articulated robots covered by rectangles

25 October 2026 · Repository `/media/4tbssd/jeffrey/G-NTFields` (branch `test1`, nothing committed)

Summary for the implementing agent

- **Goal.** Plan motions in tight spaces for a robot with joints $q \in \mathbb{R}^n$, using a learned travel-time field (metric NTFields or a homotopy head). Make each planner step safe by filtering it piece by piece over the robot's geometry.
- **Method.** Cover every link with rectangles (2-D) or boxes (3-D). Each box face f has its own clearance speed $s_f(q)$ and joint-space gradient $g_f = J_l(q, x_f)^\top n_f / m$. Replace each planner step Δq by the closest step $\Delta q'$ that satisfies $g_f^\top \Delta q' \geq -\alpha\,(s_f - s_{\text{floor}})$ for every face, plus the joint limits.
- **Evidence.** On rigid bodies this filter raised success from 69.7% to 85.1% (sofa, corridor width 30) and from 69.8% to 92–95% (Gibson cube). The same filter built on one speed for the whole body added only 0–7 points.
- **New for robots.** The kinematic Jacobian, a joint metric, joint limits, optional self-collision pairs, and a check after each step.
- **Plan.** Unit tests first, then a planar 3-link arm, then the UR5e arm already in the repo (Section 7). Files to reuse are in Section 8.

1 Background: idea 1 on rigid bodies

1.1 Clearance as a minimum over pieces

The fields are trained on the clearance speed $S(x) = \text{clip}(d(x)/m,\ o/m,\ 1)$. Here d is the distance from the scene to the body, m the margin and o the offset. The distance to a body is the minimum over its boundary pieces, so

$$S(x) = \min_i s_i(x), \qquad s_i(x) = \text{clip}(d_i(x)/m,\ o/m,\ 1),$$

where d_i is the distance from the scene to piece i alone. The pieces were the 8 edges of the planar sofa (point-to-segment distances) and the 6 rectangular faces of a box in Gibson (point-to-rectangle distances). S has a kink wherever the limiting piece changes. Each s_i is smooth across that switch.

1.2 The step filter

Idea 1 leaves the field alone and edits only the step the planner takes. After MPPI picks its step Δx, the filter solves

$$\min_{\Delta x'} \|\Delta x' - \Delta x\|^2 \quad \text{s.t.} \quad g_i^\top \Delta x' \ \geq\ -\alpha\,\big(s_i(x) - s_{\text{floor}}\big) \quad \text{for every piece } i, \qquad g_i = \nabla_x s_i(x).$$

To first order this is $s_i(x + \Delta x') \geq (1-\alpha)\, s_i(x) + \alpha\, s_{\text{floor}}$. In one step, a piece closes at most a fraction α of its gap to the floor; it is a discrete-time control barrier condition per piece. At a kink both limiting pieces keep their own half-space, so the step that lowers either one is removed. A model of S alone has one smoothed gradient there.

Solver. Hildreth's dual coordinate ascent, 30 sweeps, batched over cases. Per row i: $\lambda_i \leftarrow \max\big(0,\ \lambda_i + (h_i - g_i^\top w)/\|g_i\|^2\big)$ and $w \leftarrow w + \Delta\lambda_i\, g_i$, with $h_i = -\alpha(s_i - s_{\text{floor}})$ and w starting at Δx.

Code. `EdgeGuard` in `models/regular_planning/graph_method_triangle/edge_guard.py`:

- `heads_grad` returns the speeds and gradients.
- `filter` does the projection.
- `rollout_cost` is an optional barrier that only reweights MPPI samples: $w \sum_h (\mathrm{relu}(s_{\mathrm{safe}} - S)/s_{\mathrm{safe}})^2$.
- `summary` logs how often the filter acts and how much it changes the step.

The hook is in `models/regular_planning/graph_homotopy/scripts/plan_eval_homotopy.py`: `step_prior = GUARD.filter(XP[:, 0:DIM], step_prior)` right after MPPI picks its step. The filter works on any field (`-kind graph` or `-kind metric`). For the face guards, speeds below the offset keep falling linearly ($s = \min(d/m, 1)$, `lower_clip` off). A face that got too close is therefore pushed back out instead of losing its constraint.

Settings. Sofa: $\alpha = 1$, $s_{\mathrm{floor}} = 0.06$. Gibson: $\alpha = 0.5$, $s_{\mathrm{floor}} = 0.15$, tuned on cases 700–999 of seed 0. The Gibson settings do not use the barrier.

Speed sources. There are three:

- *Exact*: piece distances to the scene point cloud or to the true walls.
- *Learned per piece*: a net with one output per piece (`EdgeSpeedNet`, `FaceSpeedNet`).
- *One-speed*: the control, one net of S that gives a single constraint from ∇S.

1.3 Results

Task (pieces)	Field	Alone	per-piece exact	per-piece learned	one-speed learned
Sofa, corridor 30 (8 edges)	homotopy head hminL	69.7 ± 1.8	85.1 ± 3.9	82.5 ± 3.4	67.9 ± 1.2
Sofa, corridor 30 (8 edges)	metric NTFields	56.0 ± 0.7	56.7 ± 0.8	56.6 ± 0.8	–
Gibson cube (6 faces)	metric NTFields (TD + normal)	69.8 ± 3.6	92.3 ± 2.0	94.5 ± 1.5	74.3 ± 3.3
Gibson cube (6 faces)	homotopy head hminL	84.6 ± 2.1	91.2 ± 1.5	91.5 ± 1.5	87.4 ± 1.6
Gibson box (6 faces)	metric NTFields (TD + normal)	88.1 ± 1.9	94.3 ± 1.8	94.7 ± 1.9	89.4 ± 1.9
Gibson box (6 faces)	homotopy head hminL	74.0 ± 1.4	91.2 ± 1.2	89.3 ± 1.2	81.1 ± 0.4

Success %, mean $\pm$ sd over 3 seeds (sofa) or 6 seeds (Gibson; one-speed filter 3 seeds). Sofa exact is the guard tagged `gPF`, measured against the corridor walls. Sofa learned is `gQF` / `gQMF`. Gibson exact is `gXF`, measured against the scene cloud. Gibson learned is `gEF` / `gMF`. Sources: `results/sofa2d/summary/sofa_summary.json` and `results/gibson_faces/summary/gibson_faces_summary.json`.

Lessons that carry over:

1. **The per-piece filter helps every field that knows the route.** It adds 15 points on the sofa and 7 to 25 points on Gibson. The learned per-piece nets do as well as exact geometry.
2. **One speed for the whole body is not enough.** That filter adds 0–7 points, and on the sofa it doubles the stalls (non-convergence 15.5% $\rightarrow$ 31.6%). On the sofa both nets fit S to about 0.3%, so the gap comes from splitting into pieces, not from the fit.
3. **The filter fixes steps, not routes.** Metric NTFields on the sofa has no way around the corner (0.1% of corner pairs succeed). The filter turns its collisions into stalls (collisions 42.8% $\rightarrow$ 4.4%), but success stays at 56%. With a field that knows the route (hminL with per-edge features, idea 2), the exact filter reaches 100.0% $\pm$ 0.0.

2 Approximating the robot with rectangles

2.1 The cover

Each link l carries boxes $B_{l,1}, \ldots, B_{l,k_l}$ fixed in the link frame; a planar robot uses rectangles. Each box adds its 6 rectangular faces (a rectangle adds its 4 edges). The robot's body is the union of the boxes. Its clearance is the minimum over all faces of all boxes, because the closest point of a box to an outside obstacle lies on one of its faces. The boxes should *contain* each link's collision mesh. Then box clearance $\leq$ true clearance, and the filter errs on the safe side.

2.2 Fitting procedure (suggested)

1. Load each link's collision mesh in its own link frame. For the UR5e these are `meshes/collision/*.stl` (Section 7).
2. Fit an oriented bounding box that contains all mesh vertices: the PCA axes, then the minimum volume over rotations about the principal axes.
3. Measure the over-cover: the largest distance from the box surface to the mesh, and the ratio of box volume to convex-hull volume.
4. If the over-cover is large next to the task's gaps (compare it with the margin m), split the link along its long axis into 2–3 boxes and refit each part.
5. Inflate each box by a small δ to cover mesh tessellation error. Store centre, axes and half sizes per box.
6. Convert each box to faces (c, u, v, a, b): centre, unit in-plane axes with $u \times v$ the outward normal, and half sizes $a \geq b$. `faces_from_mesh` in `face_speed.py` does this from a box mesh, or build them directly from the box parameters.

2.3 Rectangles, spheres or capsules

The filter only needs a speed and a gradient per piece, so any cover works. The piece type sets how well the cover fits and how many pieces there are. The repo's UR5 labeller already covers the 6 moving links with 29 spheres: shoulder 1, upper arm 8, forearm 9, wrists 4, 4 and 3, with radii 0.043–0.100.

Piece	Distance	Fits well	UR5e pieces	Existing code
Box face	point to rectangle	flat parts: grippers, tools, carried objects, housings	36 (1 box/link)	`FaceGeometry`, `FaceSpeedNet` (`face_speed.py`)
Sphere	point distance $-$ radius	round, compact parts	29 (existing)	`dataprocessing/speed_sampling_arm_normal.py` (labels only)
Capsule	point to segment $-$ radius	long cylindrical links (upper arm, forearm)	1–2 per link	`edge_speed.py` (sofa line cells are capsules)

Recommendation: use rectangles as the main method. Their flat faces have constant normals, and the per-face code and nets already exist. Run the existing sphere cover as a baseline. Try capsules if boxes over-cover the round links too much.

2.4 Ties and overlaps

At an edge or corner contact, two or three faces of one box are equally close. Their constraints nearly coincide, and Hildreth handles the duplicates. For this reason the Gibson nets' tie-aware limiting-face

accuracy (84.5% cube, 79.3% box) is the meaningful one, not the strict argmin (≈41%). Boxes of adjacent links overlap at the joint. This does not matter for scene clearance; leave adjacent pairs out of any self-collision check.

3 The filter in joint space

The only new part is the gradient: each face's speed reaches the joints through the kinematics. Everything else is the Gibson filter with q in place of the box pose.

Placement. Forward kinematics gives link l the pose $(R_l(q), p_l(q))$. A face f of a box on link l, stored in the link frame, sits at

$$c_f(q) = R_l(q)\, c_f + p_l(q), \qquad u_f(q) = R_l(q)\, u_f, \qquad v_f(q) = R_l(q)\, v_f.$$

Speeds. With E the scene point cloud,

$$d_f(q) = \min_{p \in E} \operatorname{dist}\big(p,\ \operatorname{rect}_f(q)\big), \qquad s_f(q) = \min\big(d_f(q)/m,\ 1\big), \qquad S(q) = \min_f s_f(q).$$

Gradients. Let x_f be the closest point on face f and n_f the unit vector from its nearest scene point to x_f. The point x_f moves with link l, so

$$g_f = \nabla_q s_f(q) = \frac{1}{m}\, J_l(q, x_f)^\top n_f,$$

where $J_l(q, x)$ is the $3 \times n$ Jacobian of a point fixed to link l. Its columns for joints after link l are zero, so a forearm face never asks the wrist to move.

Implementation. `FaceGeometry.dist_grad` already fixes the nearest scene point (argmin), recomputes the point-rectangle distance differentiably and calls autograd. Put the forward kinematics in that autograd graph and it returns g_f directly. pytorch_kinematics is in the container image; the UR5 labeller already differentiates its sphere centres through forward kinematics. Take gradients with respect to the coordinates the planner steps in. `tests/arm_plan_stat.py` defines a joint scale `scale = np.pi/0.5`; check how its planner coordinates map to URDF angles.

Projection. The joint-space filter takes a joint metric W and adds the joint limits as half-spaces of the same form:

$$\min_{\Delta q'} (\Delta q' - \Delta q)^\top W\, (\Delta q' - \Delta q) \quad \text{s.t.} \quad \begin{cases} g_f^\top \Delta q' \ \geq\ -\alpha\, \big(s_f(q) - s_{\text{floor}}\big) & \text{every face } f, \\ e_k^\top \Delta q' \ \geq\ -\alpha_q\, (q_k - q_{\min,k}) & \text{every joint } k, \\ -e_k^\top \Delta q' \ \geq\ -\alpha_q\, (q_{\max,k} - q_k) & \text{every joint } k. \end{cases}$$

Hildreth's update with W: $\lambda_i \leftarrow \max\big(0,\ \lambda_i + (h_i - g_i^\top w)/(g_i^\top W^{-1} g_i)\big)$ and $w \leftarrow w + \Delta\lambda_i\, W^{-1} g_i$. With $W = I$ this is the current `EdgeGuard.filter` line for line.

Self-collision (optional). Treat each pair of boxes (A, B) on non-adjacent links a, b as another piece. Compute $d_{AB}(q)$ from the closest points x_A, x_B: separating-axis test for overlap, then the minimum over face pairs or GJK. The gradient is

$$g_{AB} = \frac{1}{m}\big(J_a(q, x_A) - J_b(q, x_B)\big)^\top n_{AB},$$

with n_{AB} the unit vector from x_B to x_A. The constraint has the same form as a face.

One planning step (pseudo-code; `B` = batch of cases, `n` = joints, `F` = faces):

```
# q: (B, n) current joints; dq: (B, n) step chosen by the planner (MPPI)
T      = fk(q)                              # world pose of every link, batched
```

```
faces  = place_faces(cover, T)              # (B, F, 11) every box face in the world
s, G   = face_speeds_and_grads(q, faces)    # (B, F), (B, F, n): exact or learned
rows   = faces with s < 1                   # beyond the margin: G = 0, always satisfied
h      = -alpha * (s - s_floor)             # row i:  G[i] . dq' >= h[i]
append joint-limit rows (and self-collision rows) to (G, h)
w, lam = dq.clone(), zeros(rows)
for sweep in range(30):                     # Hildreth, Gauss-Seidel over rows
    for i in rows:
        gW   = W_inv @ G[i]
        new  = max(0, lam[i] + (h[i] - G[i] . w) / (G[i] . gW))
        w   += (new - lam[i]) * gW
        lam[i] = new
dq_safe = w
# new for arms: check the linear prediction and back off
for tries in range(3):
    if all faces f: s_f(q + dq_safe) >= (1-alpha)*s_f(q) + alpha*s_floor - tol: break
    dq_safe *= 0.5
q = q + dq_safe     # dq_safe is also the planner's prior for the next step
                    # (as in plan_eval_homotopy.py)
```

Learned per-face speeds (optional path). In face mode, `FaceSpeedNet` reads only a face's world placement: its centre, $nn^\top$ and $uu^\top$, 15 numbers. Faces of the same size share one regressor (`face_groups`). The net therefore learns the scene once per rectangle size, from rectangle placements, not from joint configurations. Forward kinematics feeds it at plan time, so the model grows with the number of distinct rectangle sizes, not with the number of joints.

`forward_tR` currently places the net's own faces from one body pose. For an arm, add a method that takes per-face world placements (c, n, u) directly.

Training data. Rectangle placements in the robot's workspace:

- Centres uniform in the reachable volume, plus extra samples in the clearance band (the Gibson `_x1` configs have 4× extra band data).
- Orientations uniform, or taken from forward kinematics of random q.
- Labels: exact d_f from `FaceGeometry`.
- Gibson round-2 recipe: grid encoding at resolutions 16–256 and 40k steps, trained with `models/regular_planning/gibson_faces/scripts/train_face_speed.py`.

Gibson quality to expect: relative speed error 3.6% (cube) and 8.3% (box), against 13% and 13.2% for one net of S. Gradient cosine 0.96 and 0.89, against 0.62 and 0.76.

4 Why per-face matters more for an arm

- **Several contacts, one constraint each.** In a tight space an arm is close to obstacles with several links at once. Picture the elbow near a side wall, the wrist under a shelf and the gripper near the back. Each contact is its own half-space of safe joint steps, and the planner needs their intersection. A single $S(q)$ gives one half-space: the gradient of whichever piece is lowest now, or, for a learned net, a blend of the pieces near a switch. A step that clears the elbow can then push the wrist into the shelf.
- **The gradients point different ways in joint space.** A forearm face's gradient is zero on the wrist joints; a gripper face's gradient involves all joints. A blend of the two frees neither.

- **The limiting piece switches more often.** One box per UR5e link gives 36 faces, against 8 edges for the sofa and 6 faces for the box. S has more kinks along a path, and kinks are where the sofa's one-speed filter stalled.
- **Ties inside one box are harmless** (Section 2.4). The ties that matter are between links.

5 Design choices

Choice	Options	Start with	Why
Cover	1 box per link; split long links; extra boxes for gripper or tool	1 box per link	simplest; split where the over-cover is comparable to m
Constraint level	whole robot (one S); per box; per face	per face	matches Gibson; the other two are the ablations
Speed source	exact against the cloud; learned per-face net	exact	cloud available at plan time; on Gibson learned $\approx$ exact
Joint metric W	I; per-joint weights; swept motion $\sum_l J_l^\top J_l$	I	equals current code; try swept motion second
α, s_{floor}	sweep $\alpha \in \{0.25, 0.5, 1\}$, $s_{\text{floor}} \in \{0.06, 0.15, 0.25\}$	0.5, 0.15	Gibson values; tune on a held-out split
Margin m, offset o	from the speed labels	the labels' values	the filter must use the m, o of the speeds it reads
Joint limits	rows in the same QP	on	free; stops the filter pushing past a limit
Self-collision	box pairs on non-adjacent links	off in E1	add in E2 if folded poses occur
Barrier rollout cost	on / off	off	not part of the Gibson settings
Post-step check	halve up to 3 times	on	arm lever arms break the linear model sooner

UR5 labeller values (`dataprocessing/speed_sampling_arm_normal.py`): limit 0.5, $m = \text{limit}/10 = 0.05$, $o = m/10 = 0.005$ in its normalised units. A known pitfall from the sofa: the test-set metadata had offset 0.02 while the labels used 0.001. The guard now takes m and o from the speed-net checkpoint.

6 Risks and how to catch them

1. **Stalls.** The filter removes the part of a step that closes clearance. When the field points straight into an obstacle, the projected step shrinks to near zero. Seen so far: 31.6% non-convergence with the one-speed filter (sofa), and 20–30% with a dense 4M-point cloud filter on Gibson. *Catch:* report non-convergence apart from collisions, and log `filter_active_frac` and `filter_rel_change` (`EdgeGuard.summary`). *Mitigate:* MPPI keeps sampling other steps; optionally add a small step along the null space of the active constraints.
2. **No route in the field.** The filter cannot create a route the field lacks; it turns collisions into stalls. *Catch:* compare each field's success on the tight pairs without the filter first.
3. **Over-cover.** Boxes on cylindrical links can close gaps the real arm fits through. *Catch:* count pairs that have a collision-free path for the true meshes but where the boxes collide. *Fix:* split boxes or use capsules.
4. **Linearisation.** $g^\top \Delta q$ is first order. A step at the base joint moves the far links a lot, so s_f can drop more than predicted. *Catch:* log the per-step violation. *Fix:* the post-step check halves the step.

5. **Distribution shift of the learned net.** Face placements produced by an arm differ from those of a free box. Train on placements in the arm's workspace and validate on placements from forward kinematics.
6. **Cost.** Per step: one forward-kinematics call, the face speeds, and about 30 $\times$ (active rows) Hildreth updates. The current code loops over rows in Python. With 36–60 faces, drop faces with $s_f = 1$ first, or vectorise over the active rows.
7. **Self-grading.** Check collisions in evaluation with the true link meshes (or the labeller's spheres), never with the boxes alone.

7 Experiment plan

E0 — unit tests (`tests/test_arm_faces.py`)

- Every box contains its link's collision mesh (all vertices inside, after δ).
- The minimum over faces equals a direct box-to-cloud distance for random q.
- g_f matches central finite differences in q (relative error $< 10^{-3}$ where d_f is smooth).
- With $W = I$ and one free rigid body, the new filter reproduces `EdgeGuard.filter`.
- The projected step satisfies every row to 10^{-5}.

E1 — planar 3-link arm through a slot (3-D C-space)

- **Robot.** Links are rectangles, 4 edges each, using the sofa edge code. Speeds are exact.
- **Field.** Train metric NTFields on the arm's own labels; `Model` in `models/regular_planning/metric_arm` takes the dimension as an argument. Or train a homotopy head.
- **Oracle (optional).** Fast marching on a 3-D joint grid gives an exact field, which separates field errors from step errors. scikit-fmm is not in the container image.
- **Variants.** Field alone; with the one-speed filter; with the per-link filter (one constraint per rectangle); with the per-edge filter.

E2 — UR5e (6 DoF) in the repo's arm pipeline

- **Robot data.** `ur5e.urdf`, `meshes/collision/*.stl`, the sphere cover `meshes/sphere/sphere/*.npy` and a scene `realpc.obj`. A copy is at `/media/4tbssd/jeffrey/ntrl-demo/datasets/arm/UR5`. The code expects `datasets/arm/UR5` inside the repo, which is missing in this checkout.
- **Labels.** `dataprocessing/speed_sampling_arm_normal.py` computes sphere-cover distances with a kd-tree. It needs `torch_kdtree`, which the container image lacks.
- **Field.** The trainer `train/regular_planning/train_arm.py` trains `models/regular_planning/metric_arm`: `Model(ModelPath, DataPath, 6, source, device)`.
- **Planner.** MPPI in `tests/arm_plan_stat.py`: 200 steps, 50 samples, horizon 5, step 0.015. *Hook:* after `dP_prior = (weight@dP[:,0,:])`, set `dP_prior = guard.filter(XP[:,0:6], dP_prior)`, before `XP[:,0:6] = dP_prior + XP[:,0:6]`. This is the same place as in `plan_eval_homotopy.py`.
- **Not found.** The trained model and scene folders named in `tests/arm_plan_stat.py` (`Experiments/UR5/arm_09_19_01_14`, `datasets/arm/Auburn`, `Spotswood`) are not in this checkout or in ntrl-demo. A Gibson Auburn mesh is at `/media/4tbssd/jeffrey/ntrl-demo/datasets/gibson/Auburn`.
- **Variants.** Field alone; one-speed filter; per-sphere filter (existing cover); per-box filter; per-face filter, exact; per-face filter, learned.

- **Cases.** At least 1000 start–goal pairs per scene. Define a *tight* subset before running: pairs where the unfiltered field collides, or where the straight joint path's clearance falls below the margin. Tune α and s_{floor} on a held-out split, as Gibson used cases 700–999 of seed 0.
- **Metrics.** Success: goal reached within tolerance with no collision of the true meshes. Also collision %, non-convergence %, success on the tight subset, filter-active fraction, mean relative step change, and milliseconds per step. Use 3 seeds per field.
- **Acceptance (suggested).** On the tight subset, the per-face filter beats both the field alone and the one-speed filter by more than 2 sd over seeds. Its non-convergence is no more than 5 points above the field alone.

8 Implementation notes

Reuse:

- `models/regular_planning/graph_method_triangle/edge_guard.py`: `EdgeGuard` (`filter`, `heads_grad`, `rollout_cost`, `summary`).
- `models/regular_planning/graph_method_triangle/face_speed.py`: `faces_from_mesh`, `place_faces`, `point_rect_dist`, `FaceGeometry` (exact radius-gather distances, `dist_grad`), `FaceSpeedNet`, `load_face_speed_net`.
- `models/regular_planning/graph_method_triangle/edge_speed.py`: planar segments, `EdgeGeometry`, `EdgeSpeedNet` (for E1).
- `models/regular_planning/graph_homotopy/scripts/plan_eval_homotopy.py`: the guard flags (`-guard-alpha`, `-guard-floor`, `-guard-sweeps`, `-guard-lower-clip`) and the hook. Do not change its defaults; other queues run it.
- `models/regular_planning/gibson_faces/scripts/`: `preprocess_faces.py` (face labels and configs) and `train_face_speed.py` (per-face trainer; `evaluate()` scores body speed, the gradient of the minimum and limiting-face accuracy).
- The UR5 pipeline: `dataprocessing/speed_sampling_arm_normal.py`, `train/regular_planning/train_arm.py`, `models/regular_planning/metric_arm/`, `tests/arm_plan_stat.py`.

Suggested new code:

- `models/regular_planning/arm_faces/box_cover.py`: fit boxes and build their faces.
- `arm_geometry.py`: forward-kinematics placement, exact per-face speeds and joint gradients.
- `arm_guard.py`: the joint-space filter with W, joint limits and the post-step check.
- `tests/test_arm_faces.py`: the E0 tests.

Machine:

- Python runs in the docker image `pytorchserver:latest`. It has torch, pytorch_kinematics and igl; it lacks torch_kdtree, trimesh and scikit-fmm.
- Mount the repo at `/workspace` and run as your own user, since root-owned outputs break later writes:

```
docker run --rm --gpus '"device=N"' -u $(id -u):$(id -g) \
    -v $PWD:/workspace -w /workspace pytorchserver:latest python ...
```

- The four GPUs are shared with other users' training jobs; check free memory before launching.
- Nothing in the repo is committed on branch `test1`, so do not rely on git history for these files.